

Feature Ticket List

Planetic Web Automated Website, Domain, Hosting, DNS, Billing & Client Portal Platform

Project Assumptions

Build the platform as a Laravel application deployed on cPanel under:

`https://planeticweb.com`

Core stack:

- Laravel
- MySQL
- Laravel Blade
- Tailwind CSS
- Alpine.js
- Filament Admin Panel
- Stripe Checkout and Webhooks
- NameSilo API as primary registrar
- Namecheap Domain API as backup registrar
- Namecheap WHM API for hosting account automation
- Cloudflare API for DNS automation
- Namecheap cPanel Email SMTP with Jellyfish filtering
- Laravel database queues
- Laravel scheduler through cPanel cron

Priority labels:

- **Must-have for launch:** Required for MVP launch.
 - **Should-have:** Important but can launch without it if necessary.
 - **Nice-to-have:** Useful later enhancement.
-

Phase 1 — Project Foundation

Ticket 1: Set Up Laravel Project Foundation

Priority: Must-have for launch

Description:

Create the base Laravel application for Planetic Web. Configure the project for cPanel deployment,

MySQL, Laravel Blade, Tailwind CSS, Alpine.js, Vite, database sessions, database cache, and database queue.

Build Requirements:

- Install Laravel.
- Configure `.env.example`.
- Configure MySQL connection.
- Configure database sessions.
- Configure database cache.
- Configure database queue.
- Install and configure Tailwind CSS.
- Install and configure Alpine.js.
- Configure Vite build pipeline.
- Create base layout files for public pages, auth pages, customer dashboard, and admin area.
- Ensure app works under `https://planeticweb.com`.

Acceptance Criteria:

- Laravel app loads successfully.
- Tailwind CSS compiles successfully.
- Alpine.js works on frontend.
- Database connection works.
- Sessions use database driver.
- Cache uses database driver.
- Queue uses database driver.
- `.env.example` includes all required environment variables.
- App can be deployed with Laravel public directory as web root.

Dependencies:

None.

Ticket 2: Configure Environment and Service Config Files

Priority: Must-have for launch

Description:

Create dedicated configuration files for Stripe, WHM, Cloudflare, domain registrar, billing, and email settings.

Build Requirements:

Create these files:

```
config/stripe.php
config/whm.php
config/cloudflare.php
```

```
config/domain.php
config/billing.php
```

Each config file must read values from `.env`.

Acceptance Criteria:

- Stripe config reads public key, secret key, webhook secret, and currency.
- WHM config reads host, port, username, token, default package, and server IP.
- Cloudflare config reads API token, account ID, SSL mode, and proxy settings.
- Domain config supports `namesilo` and `namecheap`.
- Billing config supports grace period and renewal reminder days.
- No secret key is hardcoded in the codebase.

Dependencies:

Ticket 1.

Ticket 3: Create Core Database Migrations

Priority: Must-have for launch

Description:

Create all core database tables required for customers, orders, products, payments, domains, hosting accounts, Cloudflare zones, DNS records, website projects, support tickets, provisioning jobs, notification logs, audit logs, coupons, and subscriptions.

Build Requirements:

Create migrations for:

```
users
roles
role_user
products
product_prices
hosting_packages
carts
cart_items
orders
order_items
payments
invoices
subscriptions
domains
domain_contacts
hosting_accounts
cloudflare_zones
dns_records
```

```
provisioning_jobs
website_projects
website_project_assets
support_tickets
support_ticket_messages
notification_logs
audit_logs
coupons
coupon_redemptions
```

Acceptance Criteria:

- All migrations run successfully.
- Foreign keys are correctly added.
- `user_id` exists on all customer-owned records.
- Status columns exist where needed.
- Monetary values use decimal fields.
- JSON columns are used for metadata, nameservers, pages, and reference websites.
- Tables support soft deletes where appropriate for important records.

Dependencies:

Ticket 1.

Ticket 4: Create Laravel Models and Relationships

Priority: Must-have for launch

Description:

Create Eloquent models for all core database tables and define relationships clearly.

Build Requirements:

Create models for:

```
User
Role
Product
ProductPrice
HostingPackage
Cart
CartItem
Order
OrderItem
Payment
Invoice
Subscription
Domain
DomainContact
```

```
HostingAccount
CloudflareZone
DnsRecord
ProvisioningJob
WebsiteProject
WebsiteProjectAsset
SupportTicket
SupportTicketMessage
NotificationLog
AuditLog
Coupon
CouponRedemption
```

Acceptance Criteria:

- User has many orders, domains, hosting accounts, invoices, subscriptions, website projects, and support tickets.
- Order has many order items and provisioning jobs.
- Domain belongs to user and has one Cloudflare zone.
- Hosting account belongs to user, domain, and hosting package.
- Cloudflare zone belongs to domain and has many DNS records.
- Website project belongs to user and order.
- Support ticket has many messages.
- Models use guarded/fillable protection.
- Sensitive fields are hidden where appropriate.

Dependencies:

Ticket 3.

Ticket 5: Add Seeders for Roles, Products, Prices, and Hosting Packages

Priority: Must-have for launch

Description:

Create default seeders for roles, products, product prices, and WHM hosting package mappings.

Build Requirements:

Seed roles:

```
super_admin
technical_admin
billing_manager
support_staff
website_developer
customer
```

Seed products:

```
Complete Bespoke Website
Starter Hosting
Business Hosting
Pro Hosting
Agency / Ecommerce Hosting
Domain Registration
Website Maintenance
```

Seed hosting package mappings:

```
Starter Hosting → planetic_starter
Business Hosting → planetic_business
Pro Hosting → planetic_pro
Agency Hosting → planetic_agency
```

Acceptance Criteria:

- Seeders run successfully.
- Products appear in database.
- Prices are stored in GBP.
- Hosting package records map to correct WHM package names.
- Admin user can be created from seeder or command.

Dependencies:

Ticket 3, Ticket 4.

Phase 2 — Authentication, Roles, and Security

Ticket 6: Build Customer Authentication

Priority: Must-have for launch

Description:

Build customer registration, login, logout, forgot password, reset password, and email verification.

Build Requirements:

- Register with name, email, password, phone, company name.
- Login with email and password.
- Email verification required before sensitive dashboard access.
- Forgot/reset password flow.
- Session regeneration after login.
- Login rate limiting.

Acceptance Criteria:

- Customer can register.
- Customer receives verification email.
- Customer cannot access dashboard features until email is verified.
- Customer can log in and log out.
- Customer can reset password.
- Repeated failed login attempts are rate-limited.
- Passwords are hashed securely.

Dependencies:

Ticket 1, Ticket 3, Ticket 4.

Ticket 7: Implement Role-Based Access Control

Priority: Must-have for launch

Description:

Implement role-based permissions for customer, support staff, website developer, billing manager, technical admin, and super admin.

Build Requirements:

- Add role assignment.
- Add middleware for admin access.
- Add policies for customer-owned resources.
- Customers can only access their own records.
- Staff can only access permitted sections.
- Super admin can manage everything.

Acceptance Criteria:

- Customer cannot access admin dashboard.
- Customer cannot access another customer's records.
- Support staff cannot manage billing settings.
- Website developer cannot view full billing records.
- Billing manager cannot change API keys.
- Technical admin can retry provisioning and manage hosting/DNS status.
- Super admin can manage all records.
- Unauthorized access returns safe 403 or 404 response.

Dependencies:

Ticket 6.

Ticket 8: Add Row-Level Ownership Policies

Priority: Must-have for launch

Description:

Implement row-level security rules at Laravel policy/controller/query level so customers only access their own data.

Build Requirements:

Protect:

```
orders
invoices
payments
subscriptions
domains
domain_contacts
hosting_accounts
cloudflare_zones
dns_records
website_projects
website_project_assets
support_tickets
support_ticket_messages
```

Acceptance Criteria:

- Every customer-facing query checks `user_id`.
- A customer cannot access another customer's invoice by changing URL ID.
- A customer cannot access another customer's domain.
- A customer cannot access another customer's support ticket.
- Internal notes are hidden from customers.
- Unauthorized records return 404 or access denied.
- Tests confirm ownership protection.

Dependencies:

Ticket 7.

Ticket 9: Add Audit Logging

Priority: Must-have for launch

Description:

Create audit logging for important admin and system actions.

Build Requirements:

Log:

- Admin login
- Product/price changes

- Order status changes
- Invoice status changes
- Hosting suspension/unsuspension
- Provisioning retry
- Domain status update
- DNS record changes
- Role changes
- User status changes

Acceptance Criteria:

- Audit log records actor, action, entity type, entity ID, old values, new values, IP address, user agent, and timestamp.
- Super admin can view audit logs.
- Normal customers cannot view audit logs.
- Audit logs cannot be deleted from normal admin UI.

Dependencies:

Ticket 7.

Phase 3 — Public Website and Design System

Ticket 10: Implement Global Design System

Priority: Must-have for launch

Description:

Implement the frontend design system using Planetic-style navy, blue, and cyan colours with WCAG AA accessibility standards.

Build Requirements:

Use these core colours:

```
Primary dark: #07111F
Primary navy: #0B1B33
Primary blue: #2563EB
Primary blue hover: #1D4ED8
Accent cyan: #06B6D4
Text dark: #0F172A
Muted text: #64748B
Light background: #F8FAFC
Border: #E2E8F0
Success: #16A34A
Warning: #F59E0B
Danger: #DC2626
```

Implement:

- Buttons
- Inputs
- Cards
- Badges
- Alerts
- Modals
- Tables
- Focus states
- Typography scale
- Spacing scale

Acceptance Criteria:

- Tailwind config includes brand colours.
- Buttons have accessible contrast.
- Inputs have visible labels and focus states.
- Cards use consistent spacing and radius.
- All interactive elements have visible focus state.
- Pages are responsive on mobile, tablet, and desktop.
- No important text uses low-contrast colours.

Dependencies:

Ticket 1.

Ticket 11: Build Public Header and Footer

Priority: Must-have for launch

Description:

Create the public website header and footer.

Build Requirements:

Header:

- Logo
- Home
- Domains
- Hosting
- Website Package
- Contact
- Client Login
- Get Started CTA
- Mobile menu

Footer:

- Company information

- Services links
- Legal links
- Contact email
- Copyright
- Powered/design by Planetic Web/Planetic Solution if required

Acceptance Criteria:

- Header works on desktop and mobile.
- Mobile menu opens and closes correctly.
- Header links navigate correctly.
- Footer includes privacy policy, terms, and renewal policy.
- Keyboard users can navigate menu.
- Focus states are visible.

Dependencies:

Ticket 10.

Ticket 12: Build Home Page

Priority: Must-have for launch

Description:

Build the Planetic Web homepage with strong hosting, domain, and website package positioning.

Build Requirements:

Sections:

1. Hero with domain search
2. Trust badges
3. Hosting/domain/website service overview
4. Hosting plans preview
5. £200 website package section
6. Why choose us
7. FAQs
8. Final CTA

Acceptance Criteria:

- Home page clearly explains the service.
- Hero includes domain search input.
- CTA buttons link to domain search, hosting, and website package.
- £200 website package is visible.
- “Free domain and hosting for first year” is clearly stated.
- Page is mobile responsive.
- Page meets accessibility rules.

Dependencies:

Ticket 10, Ticket 11.

Ticket 13: Build Website Package Page

Priority: Must-have for launch

Description:

Build a dedicated landing page for the £200 bespoke website package.

Build Requirements:

Include:

- Hero offer: Complete Bespoke Website for £200
- Free first-year domain and hosting notice
- What is included
- How it works
- What customer needs to provide
- Renewal pricing note
- FAQs
- Start Now CTA

Acceptance Criteria:

- Page clearly says domain and hosting are free for the first year only.
- Page does not suggest lifetime free hosting/domain.
- CTA takes customer to domain/package selection.
- Included features are displayed clearly.
- Page is responsive and accessible.

Dependencies:

Ticket 10, Ticket 11.

Ticket 14: Build Hosting Packages Page

Priority: Must-have for launch

Description:

Build the hosting plans page showing Starter, Business, Pro, and Agency/Ecommerce packages.

Build Requirements:

Each package card should show:

- Plan name
- Short description
- Price
- Disk/storage
- Bandwidth
- Email accounts

- SSL included
- cPanel included
- Cloudflare setup
- CTA button

Acceptance Criteria:

- All seeded hosting packages appear.
- Recommended plan can be highlighted.
- CTA adds hosting package to cart or starts checkout flow.
- Pricing is pulled from backend, not hardcoded.
- Page is responsive and accessible.

Dependencies:

Ticket 5, Ticket 10, Ticket 11.

Ticket 15: Build Legal Pages

Priority: Must-have for launch

Description:

Create basic legal pages required before launch.

Build Requirements:

Pages:

- Privacy Policy
- Terms of Use
- Renewal Policy
- Refund Policy

Acceptance Criteria:

- Pages are accessible from footer.
- Renewal policy clearly explains first-year free domain/hosting and renewal charges.
- Refund policy explains domain registration limitations.
- Pages are mobile responsive.

Dependencies:

Ticket 11.

Phase 4 — Domain Search and Cart

Ticket 16: Build Registrar Interface

Priority: Must-have for launch

Description:

Create a registrar abstraction layer so the app can use NameSilo as primary registrar and Namecheap as backup.

Build Requirements:

Create:

```
RegistrarInterface
NameSiloRegistrar
NamecheapRegistrar
RegistrarResponseParser
```

Required methods:

```
checkAvailability(domain)
registerDomain(data)
renewDomain(domain, years)
getDomainInfo(domain)
updateNameservers(domain, nameservers)
```

Acceptance Criteria:

- App can switch registrar using `DOMAIN_REGISTRAR` env variable.
- Registrar services do not expose API keys.
- Registrar errors are converted into safe internal exceptions.
- NameSilo is default provider.
- Namecheap can be added/used as backup.

Dependencies:

Ticket 2.

Ticket 17: Build Domain Availability Search API

Priority: Must-have for launch

Description:

Build backend API endpoint for domain availability search.

Endpoint:

```
POST /api/domains/search
```

Build Requirements:

- Validate domain format.
- Check domain availability using registrar service.
- Return available/unavailable result.
- Return price where available.
- Return suggestions for common TLDs.
- Rate-limit the endpoint.

Acceptance Criteria:

- Valid domain returns availability result.
- Invalid domain returns validation error.
- API failure returns safe message.
- Endpoint does not expose registrar raw error to customer.
- Domain search is rate-limited.
- Response includes suggestions.

Dependencies:

Ticket 16.

Ticket 18: Build Frontend Domain Search Component

Priority: Must-have for launch

Description:

Build accessible frontend domain search component for home page and domain page.

Build Requirements:

States:

- Empty
- Typing
- Loading
- Available
- Unavailable
- Error

Acceptance Criteria:

- User can enter domain and search.
- Loading state says "Checking domain availability..."
- Available result shows Add to Cart button.
- Unavailable result shows alternatives.
- Results use `aria-live="polite"`.
- Errors are clear and accessible.

- Works on mobile and desktop.

Dependencies:

Ticket 17, Ticket 10.

Ticket 19: Build Cart System

Priority: Must-have for launch

Description:

Build cart functionality for website package, hosting package, and domain registration.

Build Requirements:

- Create cart for guest/session user.
- Attach cart to user after login/register.
- Add item to cart.
- Remove item from cart.
- Update cart totals.
- Store selected domain name.
- Store package type.
- Calculate totals server-side.

Acceptance Criteria:

- User can add website package to cart.
- User can add hosting package to cart.
- User can add domain registration to cart.
- Cart total is calculated by backend.
- Frontend cannot submit trusted price.
- Cart persists through registration/login.
- Cart page shows all selected items.

Dependencies:

Ticket 5, Ticket 17.

Ticket 20: Build Cart and Order Summary UI

Priority: Must-have for launch

Description:

Build cart and order summary pages.

Build Requirements:

Show:

- Selected package

- Selected domain
- Hosting package
- First-year included items
- Renewal notice
- Total due today
- Remove item option
- Continue to checkout button

Acceptance Criteria:

- Cart displays accurate backend totals.
- Renewal notice is clearly visible.
- £200 website package shows domain and hosting included for first year.
- User can remove items.
- Empty cart state is displayed.
- Page is mobile responsive.

Dependencies:

Ticket 19.

Phase 5 — Checkout and Stripe

Ticket 21: Build Checkout Page

Priority: Must-have for launch

Description:

Build checkout page where customer confirms order details and enters account/billing information.

Build Requirements:

- Show order summary.
- Require login/register.
- Collect billing/contact details.
- Show terms/renewal agreement checkbox.
- Start Stripe Checkout session.

Acceptance Criteria:

- Customer cannot proceed without required details.
- Customer must accept terms and renewal notice.
- Checkout uses backend-calculated total.
- Checkout button creates Stripe session.
- User is redirected to Stripe Checkout.

Dependencies:

Ticket 19, Ticket 20, Ticket 6.

Ticket 22: Build Stripe Checkout Session Creation

Priority: Must-have for launch

Description:

Create backend service to generate Stripe Checkout sessions.

Build Requirements:

- Create Stripe customer if needed.
- Create order before checkout session.
- Add order ID to Stripe metadata.
- Support one-time payment for website package.
- Support subscription mode for hosting where required.
- Return checkout URL to frontend.

Acceptance Criteria:

- Stripe session is created successfully.
- Order stores Stripe checkout session ID.
- Stripe metadata includes order ID.
- Customer is redirected to Stripe.
- No service provisioning happens at this stage.
- Errors are logged and safe message is shown.

Dependencies:

Ticket 21.

Ticket 23: Build Stripe Webhook Handler

Priority: Must-have for launch

Description:

Build secure Stripe webhook endpoint.

Endpoint:

```
POST /webhooks/stripe
```

Build Requirements:

Handle:

```
checkout.session.completed  
payment_intent.succeeded  
payment_intent.payment_failed  
invoice.paid
```

```
invoice.payment_failed
customer.subscription.created
customer.subscription.updated
customer.subscription.deleted
```

Acceptance Criteria:

- Webhook signature is verified.
- Invalid signatures are rejected.
- Duplicate events are ignored safely.
- Successful checkout marks order as paid.
- Paid order triggers provisioning queue.
- Failed payment updates order/payment status.
- Webhook events are logged.
- Services are never provisioned from frontend success page alone.

Dependencies:

Ticket 22, Ticket 24.

Ticket 24: Build Payment and Invoice Records

Priority: Must-have for launch

Description:

Store payment and invoice records based on checkout and webhook events.

Build Requirements:

- Create payment record for payment attempt.
- Update payment status after webhook.
- Create invoice record.
- Store Stripe payment intent ID.
- Store Stripe invoice ID where available.
- Store Stripe subscription ID where available.

Acceptance Criteria:

- Payment success is stored.
- Payment failure is stored.
- Invoice appears in customer dashboard.
- Admin can view payment status.
- No full card number or CVV is stored.
- Payment records are linked to user and order.

Dependencies:

Ticket 22, Ticket 23.

Ticket 25: Build Checkout Success and Cancel Pages

Priority: Must-have for launch

Description:

Build checkout success and cancel pages.

Build Requirements:

Success page:

- Tell customer payment is being confirmed.
- Show safe order status.
- Link to dashboard.

Cancel page:

- Tell customer checkout was cancelled.
- Link back to cart.

Acceptance Criteria:

- Success page does not trigger provisioning.
- Success page shows pending/provisioning status from backend.
- Cancel page does not mark order as paid.
- Customer sees helpful next step.

Dependencies:

Ticket 23.

Phase 6 — Provisioning Engine

Ticket 26: Build Provisioning Job Framework

Priority: Must-have for launch

Description:

Create queue-based provisioning engine for paid orders.

Build Requirements:

Create jobs:

```
ProvisionOrderJob
RegisterDomainJob
CreateCloudflareZoneJob
```

```
UpdateRegistrarNameserversJob
CreateWhmHostingAccountJob
CreateCloudflareDnsRecordsJob
SendProvisioningCompletedEmailJob
```

Create provisioning log records for each step.

Acceptance Criteria:

- Paid order triggers `ProvisionOrderJob`.
- Each provisioning step creates/updates a `provisioning_jobs` record.
- Failed jobs are logged with safe error message.
- Jobs are retryable.
- Jobs are idempotent.
- Admin can see provisioning status.

Dependencies:

Ticket 23, Ticket 3.

Ticket 27: Build Domain Registration Job

Priority: Must-have for launch

Description:

Build queued job to register domain after verified payment.

Build Requirements:

- Use selected registrar service.
- Register domain for 1 year.
- Use customer domain contact details.
- Enable WHOIS privacy where supported.
- Store registrar reference/order ID.
- Store registration and expiry dates.
- Mark domain active or failed.

Acceptance Criteria:

- Domain registration runs only after paid order.
- Duplicate domain registration is prevented.
- Failed registration marks order as manual review.
- Admin is notified on failure.
- Customer sees safe pending/manual review message.
- Raw registrar errors are not shown to customer.

Dependencies:

Ticket 16, Ticket 26.

Ticket 28: Build Cloudflare Zone Creation Job

Priority: Must-have for launch

Description:

Create Cloudflare zone for registered or customer-provided domain.

Build Requirements:

- Call Cloudflare API to create full zone.
- Store Cloudflare zone ID.
- Store assigned Cloudflare nameservers.
- Mark zone status pending/active.
- Handle zone already exists.

Acceptance Criteria:

- Cloudflare zone is created.
- Assigned nameservers are stored.
- Existing zone is handled safely.
- Failure marks DNS setup as manual review.
- Admin can retry failed zone creation.

Dependencies:

Ticket 26.

Ticket 29: Build Registrar Nameserver Update Job

Priority: Must-have for launch

Description:

Update domain nameservers at registrar to Cloudflare nameservers.

Build Requirements:

- Read nameservers from Cloudflare zone record.
- Send nameservers to registrar API.
- Store updated nameservers in domain record.
- Mark nameserver update status.

Acceptance Criteria:

- Registered domain gets Cloudflare nameservers.
- Failed update is logged.
- Customer sees DNS activation pending.
- Admin can retry.
- Job does not run without Cloudflare nameservers.

Dependencies:

Ticket 27, Ticket 28.

Ticket 30: Build WHM Hosting Service

Priority: Must-have for launch

Description:

Create WHM service for Namecheap reseller hosting automation.

Build Requirements:

Methods:

```
createAccount  
suspendAccount  
unsuspendAccount  
terminateAccount  
listAccounts  
getAccountInfo  
changePackage
```

Acceptance Criteria:

- WHM API token is read from `.env`.
- Create account works with package name.
- Suspend account works.
- Unsuspend account works.
- Errors are converted into safe exceptions.
- WHM credentials are never exposed to frontend.
- Unit test can mock WHM responses.

Dependencies:

Ticket 2.

Ticket 31: Build WHM Hosting Account Creation Job

Priority: Must-have for launch

Description:

Create cPanel account through WHM after payment.

Build Requirements:

- Generate safe WHM username.
- Generate secure password.
- Use selected WHM package.
- Pass customer contact email.
- Enable SPF/DKIM/DMARC where supported.

- Store WHM username, server IP, package, status, and cPanel URL.

Acceptance Criteria:

- WHM account is created for paid order.
- Duplicate WHM account creation is prevented.
- Domain already exists in WHM triggers manual review.
- Account creation failure logs error and notifies admin.
- Hosting account appears in customer dashboard.
- Admin can retry failed hosting creation.

Dependencies:

Ticket 30, Ticket 26, Ticket 5.

Ticket 32: Build Cloudflare DNS Record Creation Job

Priority: Must-have for launch

Description:

Create default DNS records in Cloudflare for the customer domain and hosting account.

Build Requirements:

Create:

```
A @ → WHM server IP, proxied
CNAME www → domain, proxied
A mail → WHM server IP, DNS only
A cpanel → WHM server IP, DNS only
A webmail → WHM server IP, DNS only
MX @ → mail.domain.com, DNS only
TXT @ → SPF record
TXT _dmarc → DMARC record
```

Acceptance Criteria:

- Website records are proxied.
- Email/cPanel records are DNS-only.
- DNS records are stored in database.
- Duplicate records are not created on retry.
- Failure is logged and retryable.
- Customer sees DNS status.

Dependencies:

Ticket 28, Ticket 31.

Phase 7 — Customer Dashboard

Ticket 33: Build Customer Dashboard Shell

Priority: Must-have for launch

Description:

Build customer dashboard layout and navigation.

Build Requirements:

Navigation:

```
Dashboard
Domains
Hosting
Billing
Website Projects
Support
Account Settings
Logout
```

Acceptance Criteria:

- Dashboard is accessible only to logged-in customers.
- Sidebar appears on desktop.
- Mobile menu works on small screens.
- Customer sees only own data.
- Layout is responsive.
- Dashboard uses design system.

Dependencies:

Ticket 6, Ticket 8, Ticket 10.

Ticket 34: Build Dashboard Summary Page

Priority: Must-have for launch

Description:

Create dashboard home page showing customer service overview.

Build Requirements:

Show:

- Active domains count
- Active hosting accounts count
- Open invoices count
- Next renewal date
- Website project status
- Support ticket summary
- Provisioning status if setup is in progress

Acceptance Criteria:

- Summary uses real customer data.
- Empty states appear for new customers.
- Pending provisioning state is clear.
- Failed/manual review states are shown safely.
- Customer cannot see another customer's data.

Dependencies:

Ticket 33.

Ticket 35: Build Customer Domains Page

Priority: Must-have for launch

Description:

Build customer domain list and detail pages.

Build Requirements:

Show:

- Domain name
- Registrar
- Registration date
- Expiry date
- Auto-renew status
- Nameserver status
- Cloudflare status
- Renewal date

Acceptance Criteria:

- Customer sees only own domains.
- Domain detail page shows safe status.
- DNS status is clear.
- Expiry date is displayed.
- Manual review/pending states are shown.
- No raw API data is shown.

Dependencies:

Ticket 33, Ticket 27, Ticket 28.

Ticket 36: Build Customer Hosting Page

Priority: Must-have for launch

Description:

Build customer hosting list and hosting detail pages.

Build Requirements:

Show:

- Hosting package
- Domain
- WHM username
- Server IP
- cPanel URL
- Account status
- Renewal date
- Disk/bandwidth limits where available

Acceptance Criteria:

- Customer sees only own hosting accounts.
- Hosting status is displayed clearly.
- Suspended accounts show reason if safe.
- cPanel link is available where configured.
- No WHM token or password is shown.
- Customer cannot edit WHM package directly.

Dependencies:

Ticket 33, Ticket 31.

Ticket 37: Build Customer Billing Page

Priority: Must-have for launch

Description:

Build customer billing area for invoices, payments, and subscriptions.

Build Requirements:

Show:

- Invoice list
- Invoice detail

- Payment status
- Subscription status
- Renewal dates
- Payment method update link placeholder

Acceptance Criteria:

- Customer sees only own invoices.
- Paid/open/failed statuses are clear.
- Invoice detail shows order items.
- No full card data is shown.
- Failed payment state shows retry guidance.
- Billing page is mobile responsive.

Dependencies:

Ticket 24, Ticket 33.

Phase 8 — Website Project Management

Ticket 38: Build Website Project Creation After Website Package Purchase

Priority: Must-have for launch

Description:

Automatically create a website project when customer purchases the £200 website package.

Build Requirements:

- Detect website package order item.
- Create website project record.
- Link project to order, user, domain, and hosting account if available.
- Set status to `information_required`.

Acceptance Criteria:

- Website project is created after paid website package order.
- Project appears in customer dashboard.
- Project appears in admin dashboard.
- Duplicate project is not created on webhook retry.

Dependencies:

Ticket 23, Ticket 26.

Ticket 39: Build Website Project Intake Form

Priority: Must-have for launch

Description:

Build customer form to submit website project details.

Build Requirements:

Fields:

- Business name
- Business description
- Industry
- Pages required
- Brand colours
- Reference websites
- Special requirements
- Contact details
- File uploads for logo/content/images

Acceptance Criteria:

- Customer can submit intake form.
- Customer can upload allowed file types.
- Dangerous file types are blocked.
- File size is limited.
- Submitted data appears in admin.
- Project status updates to `content_received`.

Dependencies:

Ticket 38, Ticket 33.

Ticket 40: Build Website Project Status View

Priority: Must-have for launch

Description:

Allow customers to view website project progress.

Build Requirements:

Statuses:

```
information_required
content_received
design_in_progress
review_required
```

```
revisions_in_progress
approved
launched
completed
on_hold
cancelled
```

Acceptance Criteria:

- Customer sees current project status.
- Customer sees next required action.
- Customer cannot directly change internal status.
- Admin can update status.
- Status updates can trigger email notifications.

Dependencies:

Ticket 38, Ticket 39.

Phase 9 — Support Tickets

Ticket 41: Build Customer Support Ticket System

Priority: Must-have for launch

Description:

Create customer support ticket system.

Build Requirements:

Customer can:

- Create ticket
- Select category
- Add subject/message
- View own tickets
- Reply to own tickets

Acceptance Criteria:

- Customer can open support ticket.
- Customer can view only own tickets.
- Customer can reply.
- Ticket messages are stored.
- Internal notes are hidden from customer.
- Ticket creation is rate-limited.

Dependencies:

Ticket 33, Ticket 8.

Ticket 42: Build Admin Support Ticket Management

Priority: Must-have for launch

Description:

Build admin support ticket management using Filament.

Build Requirements:

Admin can:

- View tickets
- Filter by status/category/priority
- Reply to customer
- Add internal notes
- Assign ticket to staff
- Close/reopen ticket

Acceptance Criteria:

- Admin can manage tickets.
- Internal notes are not visible to customer.
- Customer receives email on public reply.
- Assigned admin/staff is stored.
- Ticket status updates correctly.

Dependencies:

Ticket 41, Ticket 57.

Phase 10 — Admin Panel

Ticket 43: Install and Configure Filament Admin Panel

Priority: Must-have for launch

Description:

Install Filament Admin Panel and configure admin authentication and resources.

Build Requirements:

- Install Filament.
- Protect `/admin`.

- Allow only admin/staff roles.
- Create admin dashboard layout.
- Add widgets for revenue, orders, provisioning failures, and support tickets.

Acceptance Criteria:

- `/admin` is not accessible to customers.
- Super admin can log in.
- Admin dashboard loads.
- Role checks work.
- Basic metrics are visible.

Dependencies:

Ticket 7.

Ticket 44: Build Admin Customer Management

Priority: Must-have for launch

Description:

Create admin resource to manage customers.

Build Requirements:

Admin can:

- View customers
- Search customers
- View customer services
- View customer orders
- View customer support tickets
- Suspend/deactivate user account where needed

Acceptance Criteria:

- Admin can search by name/email/company.
- Admin can view customer profile.
- Admin cannot see customer password.
- Admin actions are audit logged.
- Customers can be activated/deactivated.

Dependencies:

Ticket 43, Ticket 9.

Ticket 45: Build Admin Product and Pricing Management

Priority: Must-have for launch

Description:

Create admin interface for products, prices, and hosting package mappings.

Build Requirements:

Admin can:

- View products
- Edit product descriptions
- Activate/deactivate products
- Edit prices
- Manage billing cycle
- Manage WHM package mapping

Acceptance Criteria:

- Products can be managed from admin.
- Pricing updates are reflected on frontend.
- WHM package mapping can be edited by Super Admin.
- Price changes are audit logged.
- Customers cannot access this area.

Dependencies:

Ticket 43, Ticket 5.

Ticket 46: Build Admin Order Management

Priority: Must-have for launch

Description:

Create admin order management resource.

Build Requirements:

Admin can:

- View orders
- Filter by status
- View order items
- View payment status
- View provisioning status
- Mark manual review notes
- Cancel order where allowed

Acceptance Criteria:

- Admin can view all orders.
- Order detail shows customer, items, total, payment, and provisioning status.
- Admin can add internal notes.
- Status changes are audit logged.

- Admin cannot accidentally mark unpaid order as completed without warning.

Dependencies:

Ticket 43, Ticket 24, Ticket 26.

Ticket 47: Build Admin Provisioning Monitor

Priority: Must-have for launch

Description:

Create admin interface to monitor and retry provisioning jobs.

Build Requirements:

Admin can view:

- Order
- Job type
- Status
- Attempts
- Error message
- Started/completed/failed time
- Retry button

Acceptance Criteria:

- Failed jobs are visible.
- Admin can retry safe jobs.
- Retry increments attempts.
- Manual review status is visible.
- Raw secrets are never shown.
- Retry action is audit logged.

Dependencies:

Ticket 26, Ticket 43.

Ticket 48: Build Admin Domain Management

Priority: Must-have for launch

Description:

Create admin resource for domain records.

Build Requirements:

Admin can:

- View domains

- View registrar
- View expiry date
- View nameservers
- View Cloudflare status
- Sync domain info
- Retry nameserver update
- Mark manual review

Acceptance Criteria:

- Admin can search domains.
- Admin can filter by status.
- Admin can retry failed registrar actions.
- Expiry dates are visible.
- Domain actions are audit logged.

Dependencies:

Ticket 27, Ticket 29, Ticket 43.

Ticket 49: Build Admin Hosting Management

Priority: Must-have for launch

Description:

Create admin resource for hosting accounts.

Build Requirements:

Admin can:

- View hosting accounts
- View WHM username
- View package
- View status
- View renewal date
- Retry account creation
- Suspend account
- Unsuspend account
- Sync status

Acceptance Criteria:

- Admin can suspend hosting.
- Admin can unsuspend hosting.
- Suspend/unsuspend calls WHM service.
- Actions are audit logged.
- Failed actions are logged and shown safely.

Dependencies:

Ticket 30, Ticket 31, Ticket 43.

Ticket 50: Build Admin Website Project Management

Priority: Must-have for launch

Description:

Create admin resource for website development projects.

Build Requirements:

Admin/developer can:

- View projects
- View intake details
- View uploaded files
- Update project status
- Add internal notes
- Request more information
- Mark project launched/completed

Acceptance Criteria:

- Projects appear after website package purchase.
- Admin can update status.
- Status updates are visible to customer.
- Uploaded files are accessible to authorized staff only.
- Status changes can trigger email notification.

Dependencies:

Ticket 38, Ticket 39, Ticket 43.

Phase 11 — Email Notifications

Ticket 51: Configure SMTP Email Sending

Priority: Must-have for launch

Description:

Configure Laravel mail to send through Namecheap cPanel Email SMTP.

Build Requirements:

Use:

```
MAIL_HOST=mail.planeticweb.com
MAIL_PORT=465
```

```
MAIL_ENCRYPTION=ssl
MAIL_USERNAME=no-reply@planeticweb.com
```

Acceptance Criteria:

- Test email sends successfully.
- Failed email is logged.
- SMTP password is only in `.env`.
- Mail from name is "Planetic Web".
- App supports fallback error logging if mail fails.

Dependencies:

Ticket 2.

Ticket 52: Build Core Email Templates

Priority: Must-have for launch

Description:

Create email templates for key customer events.

Templates:

```
Account verification
Order confirmation
Payment success
Payment failed
Domain registered
Hosting created
Provisioning manual review
Renewal reminder
Hosting suspended
Hosting reactivated
Support ticket reply
Website project status update
```

Acceptance Criteria:

- Templates use Planetic Web branding.
- Emails are responsive.
- Emails include clear next action.
- Renewal emails include renewal date and amount where available.
- Failed payment emails include retry instructions.
- No raw API errors are emailed to customer.

Dependencies:

Ticket 51.

Ticket 53: Build Notification Log System

Priority: Must-have for launch

Description:

Log all email notifications sent by the system.

Build Requirements:

Track:

- User
- Notification type
- Channel
- Recipient
- Subject
- Status
- Sent time
- Failed time
- Error message

Acceptance Criteria:

- Successful emails are logged.
- Failed emails are logged.
- Admin can view logs.
- Admin can resend failed important emails.
- Logs do not expose SMTP password.

Dependencies:

Ticket 51, Ticket 43.

Phase 12 — Renewals and Subscription Management

Ticket 54: Build Renewal Date Tracking

Priority: Must-have for launch

Description:

Track renewal dates for hosting accounts, domains, and subscriptions.

Build Requirements:

- Store renewal date for hosting.
- Store expiry date for domain.

- Store next renewal date for subscription.
- Display renewal dates in customer dashboard and admin.

Acceptance Criteria:

- Domain expiry dates display correctly.
- Hosting renewal dates display correctly.
- Customer dashboard shows next renewal.
- Admin can filter services due soon.
- Missing renewal dates are flagged for admin review.

Dependencies:

Ticket 35, Ticket 36, Ticket 37.

Ticket 55: Build Renewal Reminder Scheduler

Priority: Must-have for launch

Description:

Create scheduled job to send renewal reminder emails.

Build Requirements:

Send reminders:

30 days before renewal
14 days before renewal
7 days before renewal
3 days before renewal
1 day before renewal
On renewal day

Acceptance Criteria:

- Scheduler finds upcoming renewals.
- Reminder emails are sent.
- Duplicate reminder for same day/service is prevented.
- Notification logs are created.
- Failed emails are logged.
- Admin can see upcoming renewals.

Dependencies:

Ticket 52, Ticket 54.

Ticket 56: Build Failed Payment Handling

Priority: Must-have for launch

Description:

Handle failed Stripe payments for subscriptions and renewals.

Build Requirements:

- Listen to `invoice.payment_failed`.
- Mark subscription as past due.
- Send failed payment email.
- Start grace period.
- Show warning in dashboard.
- Notify admin.

Acceptance Criteria:

- Failed payment updates local records.
- Customer receives failed payment email.
- Dashboard shows payment issue.
- Hosting is not immediately suspended before grace period.
- Admin can see failed payment list.

Dependencies:

Ticket 23, Ticket 37, Ticket 52.

Ticket 57: Build Hosting Suspension After Grace Period

Priority: Should-have

Description:

Automatically suspend hosting after failed renewal grace period.

Build Requirements:

- Daily scheduler checks overdue unpaid hosting subscriptions.
- If grace period exceeded, call WHM suspend API.
- Update hosting status to suspended.
- Send suspension email.
- Log action.

Acceptance Criteria:

- Hosting is suspended only after grace period.
- WHM suspend API is called.
- Customer receives notification.
- Admin can see suspension reason.
- Suspension action is audit logged.
- Failed suspension is logged for manual review.

Dependencies:

Ticket 30, Ticket 56.

Ticket 58: Build Hosting Unsuspension After Payment

Priority: Should-have

Description:

Automatically unsuspend hosting after overdue payment is received.

Build Requirements:

- Listen for successful payment on overdue subscription.
- Call WHM unsuspend API.
- Update hosting status to active.
- Send reactivation email.
- Log action.

Acceptance Criteria:

- Paid overdue hosting is reactivated.
- WHM unsuspend API is called.
- Customer receives reactivation email.
- Failed unsuspension is logged.
- Admin can retry.

Dependencies:

Ticket 30, Ticket 57.

Phase 13 — DNS and Advanced Service Management

Ticket 59: Build Admin DNS Record Viewer

Priority: Should-have

Description:

Allow admin to view DNS records stored for each domain.

Build Requirements:

Admin can view:

- Record type
- Name
- Content
- TTL
- Proxied status
- Cloudflare record ID

Acceptance Criteria:

- Admin can view DNS records for each domain.
- Records show proxied/DNS-only status.
- Mail records show warning if proxied.
- Admin cannot see Cloudflare API token.

Dependencies:

Ticket 32, Ticket 43.

Ticket 60: Build Admin DNS Record Editor

Priority: Should-have

Description:

Allow technical admin to create, update, and delete Cloudflare DNS records.

Build Requirements:

- Create DNS record
- Update DNS record
- Delete DNS record
- Sync local database
- Warn before editing MX/SPF/DKIM/DMARC records

Acceptance Criteria:

- DNS record changes call Cloudflare API.
- Local database updates after successful Cloudflare response.
- Admin is warned before risky changes.
- All DNS changes are audit logged.
- Failed changes show safe error.

Dependencies:

Ticket 59.

Ticket 61: Build Customer DNS Viewer

Priority: Nice-to-have

Description:

Allow customers to view DNS records for their own domains.

Build Requirements:

- Customer can view DNS records.
- Customer cannot edit records in MVP.
- Records are simplified for non-technical users.

Acceptance Criteria:

- Customer sees only own domain DNS records.
- Customer does not see API IDs.
- Customer does not see raw Cloudflare errors.
- DNS-only/proxied status is explained clearly.

Dependencies:

Ticket 59, Ticket 35.

Ticket 62: Build Customer DNS Editor

Priority: Nice-to-have

Description:

Allow customers to edit basic DNS records safely.

Build Requirements:

Allow editing:

A
CNAME
TXT

Restrict or warn heavily for:

MX
SPF
DKIM
DMARC
mail
cpanel
webmail

Acceptance Criteria:

- Customer can only edit own domain records.
- Risky records require confirmation.
- Invalid records are rejected.
- Changes are audit logged.
- Admin can view customer DNS changes.

Dependencies:

Ticket 61.

Phase 14 — File Uploads and Asset Security

Ticket 63: Build Secure Website Project File Uploads

Priority: Must-have for launch

Description:

Allow customers to upload website project assets securely.

Build Requirements:

Allowed file types:

```
jpg
jpeg
png
webp
pdf
doc
docx
txt
```

Block:

```
php
exe
js
sh
bat
html
svg unless sanitized
```

Acceptance Criteria:

- Customer can upload allowed files.
- Blocked file types are rejected.
- File size limit is enforced.
- Files are stored outside public direct access.
- Files are downloaded through authenticated route.
- Customer can access only own files.
- Admin/developer can access assigned project files.

Dependencies:

Ticket 39, Ticket 8.

Phase 15 — Error Handling and Edge Cases

Ticket 64: Build Global Error Handling System

Priority: Must-have for launch

Description:

Create consistent error handling for frontend and backend.

Build Requirements:

Handle:

- Validation errors
- Payment errors
- Registrar errors
- WHM errors
- Cloudflare errors
- SMTP errors
- Permission errors
- Queue failures

Acceptance Criteria:

- Customers see safe, friendly messages.
- Admin sees useful but non-secret error details.
- Raw API errors are not shown to customers.
- Errors are logged.
- Failed provisioning errors appear in admin.
- 404/403/500 pages are branded and accessible.

Dependencies:

Ticket 10, Ticket 26, Ticket 43.

Ticket 65: Build Duplicate Processing Protection

Priority: Must-have for launch

Description:

Prevent duplicate orders, duplicate Stripe webhook processing, duplicate domain registration, duplicate WHM account creation, and duplicate Cloudflare zones.

Build Requirements:

- Store processed Stripe event IDs.
- Check existing domain before registration.
- Check existing hosting account before WHM create.

- Check existing Cloudflare zone before creation.
- Use database unique constraints where appropriate.

Acceptance Criteria:

- Duplicate webhook does not create duplicate services.
- Refreshing checkout success page does not provision again.
- Retried jobs do not duplicate external resources.
- Duplicate domain/hosting attempts move to manual review.
- Tests cover duplicate webhook scenario.

Dependencies:

Ticket 23, Ticket 26.

Ticket 66: Build System Health Admin Page

Priority: Should-have

Description:

Create admin page showing health status of important systems.

Build Requirements:

Show status for:

- Database
- Queue
- Scheduler
- Stripe config
- SMTP config
- Cloudflare config
- WHM config
- Registrar config

Acceptance Criteria:

- Admin can see if required config values are missing.
- Admin can see last successful queue run.
- Admin can see recent failed jobs.
- Admin can see recent failed emails.
- No secrets are shown.

Dependencies:

Ticket 43, Ticket 53.

Phase 16 — Deployment and Operations

Ticket 67: Add cPanel Deployment Documentation

Priority: Must-have for launch

Description:

Create deployment instructions for cPanel hosting.

Build Requirements:

Include:

- Folder structure
- Public directory setup
- Composer install
- NPM build or local asset build
- `.env` setup
- Database migration
- Storage link
- Config cache
- Route cache
- View cache
- Queue cron
- Scheduler cron

Acceptance Criteria:

- README includes cPanel deployment steps.
- Cron commands are documented.
- Required PHP version is documented.
- Required extensions are documented.
- Public directory security is explained.

Dependencies:

Ticket 1.

Ticket 68: Configure Laravel Scheduler and Cron Commands

Priority: Must-have for launch

Description:

Configure Laravel scheduler for renewals, syncs, and failed provisioning retries.

Build Requirements:

Scheduled commands:

```
renewals:check  
domains:sync
```

```
hosting:sync  
provisioning:retry-failed
```

Acceptance Criteria:

- Commands exist.
- Scheduler runs commands at correct times.
- cPanel cron command is documented.
- Commands log output safely.
- Commands do not expose secrets.

Dependencies:

Ticket 54, Ticket 55, Ticket 26.

Ticket 69: Configure Queue Worker for cPanel

Priority: Must-have for launch

Description:

Configure cron-based queue worker suitable for cPanel.

Build Requirements:

Use command:

```
php artisan queue:work --stop-when-empty --tries=3 --timeout=120
```

Acceptance Criteria:

- Queue jobs run from cPanel cron.
- Failed jobs are recorded.
- Queue does not require Supervisor for MVP.
- Provisioning jobs process successfully.
- Queue setup is documented.

Dependencies:

Ticket 26, Ticket 67.

Ticket 70: Build Backup and Recovery Checklist

Priority: Should-have

Description:

Create backup strategy documentation and admin checklist.

Build Requirements:

Document backups for:

- MySQL database
- Uploaded website project files
- `.env` secure copy
- Storage directory
- Logs

Acceptance Criteria:

- Backup process is documented.
- Restore process is documented.
- Backups are not stored publicly.
- Sensitive files are protected.
- Admin knows what to do before major deployment.

Dependencies:

Ticket 67.

Phase 17 — Testing and QA

Ticket 71: Add Feature Tests for Authentication and Access Control

Priority: Must-have for launch

Description:

Create automated tests for customer authentication and access protection.

Acceptance Criteria:

- Customer can register and login.
- Unverified customer cannot access restricted features.
- Customer cannot access another customer's invoice.
- Customer cannot access another customer's domain.
- Customer cannot access admin dashboard.
- Admin can access admin dashboard.

Dependencies:

Ticket 6, Ticket 7, Ticket 8.

Ticket 72: Add Feature Tests for Checkout and Stripe Webhooks

Priority: Must-have for launch

Description:

Create tests for checkout session creation and Stripe webhook handling.

Acceptance Criteria:

- Checkout session can be created.
- Order is created before Stripe redirect.
- Valid webhook marks order paid.
- Invalid webhook signature is rejected.
- Duplicate webhook does not create duplicate services.
- Failed payment updates status.

Dependencies:

Ticket 22, Ticket 23, Ticket 65.

Ticket 73: Add Unit Tests for External Service Classes

Priority: Should-have

Description:

Create unit tests for registrar, WHM, Cloudflare, and billing service wrappers using mocked responses.

Acceptance Criteria:

- NameSilo availability parser is tested.
- NameSilo registration response is tested.
- WHM create account success/failure is tested.
- Cloudflare create zone response is tested.
- Cloudflare DNS record response is tested.
- Stripe service wrapper is tested.

Dependencies:

Ticket 16, Ticket 30, Ticket 32.

Ticket 74: Manual QA Checklist Before Launch

Priority: Must-have for launch

Description:

Create manual QA checklist for launch testing.

Checklist Must Cover:

- Public pages
- Mobile responsiveness
- Accessibility
- Registration
- Login

- Domain search
- Cart
- Checkout
- Stripe test payment
- Webhook processing
- Domain registration test
- WHM hosting test account
- Cloudflare zone test
- DNS records
- Email sending
- Customer dashboard
- Admin dashboard
- Support tickets
- Renewal reminder test
- Failed payment test

Acceptance Criteria:

- QA checklist exists.
- Each item has pass/fail column.
- Launch is not approved until all must-have items pass.
- Known issues are documented.

Dependencies:

All must-have tickets.

Phase 18 — Should-Have and Nice-to-Have Enhancements

Ticket 75: Add Coupon and Discount System

Priority: Should-have

Description:

Build coupon system for fixed or percentage discounts.

Acceptance Criteria:

- Admin can create coupon.
- Customer can apply coupon.
- Coupon validates expiry and usage limit.
- Discount appears in cart and order.
- Backend calculates discount.
- Coupon usage is recorded.

Dependencies:

Ticket 19, Ticket 21, Ticket 45.

Ticket 76: Add Hosting Upgrade/Downgrade Flow

Priority: Nice-to-have

Description:

Allow customers/admin to upgrade or downgrade hosting package.

Acceptance Criteria:

- Customer can request upgrade.
- Admin can approve/change package.
- WHM `changepackage` is called.
- Billing adjustment is handled.
- Change is audit logged.

Dependencies:

Ticket 30, Ticket 37, Ticket 49.

Ticket 77: Add One-Click cPanel Login

Priority: Nice-to-have

Description:

Add secure one-click cPanel login or manage link where supported.

Acceptance Criteria:

- Customer can open cPanel from dashboard.
- Link is secure.
- No password is exposed.
- Access is limited to account owner.
- Fallback normal cPanel URL is shown if SSO is not available.

Dependencies:

Ticket 36.

Ticket 78: Add Domain Transfer Support

Priority: Nice-to-have

Description:

Allow customers to transfer existing domains into Planetic Web management.

Acceptance Criteria:

- Customer can enter domain for transfer.

- Customer can enter EPP/Auth code.
- Registrar transfer API is called if supported.
- Transfer status is tracked.
- Customer sees transfer progress.

Dependencies:

Ticket 16, Ticket 35.

Ticket 79: Add Cloudflare Cache Purge Button

Priority: Nice-to-have

Description:

Allow technical admin, and later customer, to purge Cloudflare cache.

Acceptance Criteria:

- Admin can purge cache for a domain.
- Cloudflare API is called.
- Action is audit logged.
- Customer-safe message is shown.
- Failure is logged.

Dependencies:

Ticket 28, Ticket 43.

Ticket 80: Add Website Maintenance Subscription

Priority: Nice-to-have

Description:

Add optional website maintenance product and subscription.

Acceptance Criteria:

- Maintenance product exists.
- Customer can purchase maintenance.
- Stripe subscription is created.
- Dashboard shows maintenance subscription.
- Renewal/payment status is tracked.

Dependencies:

Ticket 5, Ticket 22, Ticket 37.

Final Launch Scope Recommendation

Must-Have for Launch

The launch should include:

- Project foundation
- Database schema
- Authentication
- Role-based access
- Row-level ownership rules
- Public website
- Design system
- Website package page
- Hosting package page
- Domain search
- Cart
- Checkout
- Stripe payment
- Stripe webhook
- Payment/invoice records
- Provisioning engine
- Domain registration
- Cloudflare zone setup
- WHM hosting creation
- Default DNS records
- Customer dashboard
- Admin dashboard
- Website intake form
- Support tickets
- Email notifications
- Renewal tracking
- Renewal reminders
- Error handling
- Duplicate protection
- cPanel deployment documentation
- Queue and scheduler cron setup
- Launch QA checklist

Should-Have Soon After Launch

- Automatic suspension after grace period
- Automatic unsuspension after late payment
- Admin DNS editor
- System health page
- Backup/recovery process

Coupons
More integration tests

Nice-to-Have Later

Customer DNS editor
Hosting upgrade/downgrade
One-click cPanel login
Domain transfers
Cloudflare cache purge
Website maintenance subscription
Affiliate/reseller system
Advanced reporting
Live chat
AI website builder